



PROJECT PHASE 2 - LocalGPT

ChatGPT-Style Local AI Assistant with RAG and LLM X-Ray

1. Project Overview

Build a **local ChatGPT-style AI application** that allows users to have continuous conversations with an LLM, upload documents, ask questions about those documents, maintain conversation history, and inspect the internal LLM inference process.

The application will use the same local Hugging Face LLM from Project 1, so **no external LLM API is required**.

Core Flow

```
graph TD
    User --> ChatInterface[Chat Interface]
    ChatInterface --> ConversationMemory[Conversation Memory]
    ConversationMemory --> ContextManager[Context Manager]
    ContextManager --> DocumentRetrieval[Document Retrieval (if required)]
    DocumentRetrieval --> QwenLLM[Qwen LLM]
    QwenLLM --> StreamingResponse[Streaming Response]
    StreamingResponse --> ChatGPTStyleAnswer[ChatGPT-style Answer]
```

Optional:

```
graph TD
    Response --> LLMXRay[LLM X-Ray]
    LLMXRay --> Tokens[Tokens]
    Tokens --> Embeddings[Embeddings]
    Embeddings --> Layers[Layers]
    Layers --> Attention[Attention]
    Attention --> HiddenStates[Hidden States]
    HiddenStates --> Logits[Logits]
    Logits --> Probabilities[Probabilities]
```

The X-Ray functionality from Project 1 becomes an **advanced inspection mode** rather than the main interface.

2. Main Objective

The goal is to upgrade the existing **LLM X-Ray** into a practical **local AI assistant similar to ChatGPT**.

The system should support:

- Multi-turn conversations
- Conversation history
- Context-aware responses
- Streaming text generation
- Regenerate response
- Edit user messages
- Copy responses
- Markdown rendering
- Code formatting
- System prompt
- Temperature / Top-K / Top-P controls
- Document upload
- Question answering from documents
- Conversation-based memory
- LLM X-Ray inspection

The application should still run locally on a personal laptop.

3. Model

Use the existing model:

Qwen/Qwen2.5-1.5B-Instruct

This keeps Project 2 compatible with Project 1 and avoids requiring an external API. Project 1 already specifies this model as a locally runnable, instruction-tuned Hugging Face model.

Use:

```
AutoTokenizer  
AutoModelForCausalLM
```

from Hugging Face Transformers, as in Project 1.

4. Technology Stack

Python
PyTorch
Hugging Face Transformers
Streamlit
FAISS
Sentence Transformers
PyPDF
python-docx
Plotly
NumPy

New technologies compared with Project 1

Technology	Purpose
FAISS	Vector database / similarity search
Sentence Transformers	Document embeddings
PyPDF	Read PDF documents
python-docx	Read Word documents
Streamlit	ChatGPT-style UI
Plotly	Visualization
Transformers	Local LLM

5. Project Structure

```
LocalGPT/
├── app.py
├── model.py
├── tokenizer.py
├── chat.py
├── memory.py
├── document_loader.py
├── embeddings.py
├── vector_store.py
├── rag.py
├── xray.py
├── visualization.py
├── database.py
├── requirements.txt
├── data/
│   ├── documents/
│   └── conversations/
```

6. Implementation Steps

Step 1 — Reuse Project 1

Take the existing LLM loading and generation code.

The existing project already has:

```
app.py
model.py
tokenizer.py
visualization.py
```

and uses:

```
PyTorch
Transformers
Streamlit
Plotly
Scikit-learn
NumPy
```

as its base stack.

Do **not** rebuild the model from scratch.

Step 2 — Build ChatGPT Interface

Replace the simple single-prompt interface with a conversation interface.

Example:

LOCALGPT	
New Chat	User: Explain AI
Chat 1	AI: Artificial Intelligence
Chat 2	
Chat 3	User: Give an example AI: For example...
+ New Chat	
Upload File	Ask anything... [▶]

The interface should behave like a normal AI assistant rather than the Project 1 visualization-first interface.

Step 3 — Implement Multi-Turn Conversation

Project 1 generates a response from a single prompt.

Project 2 should maintain:

User:
What is Machine Learning?

AI:
Machine Learning is...

User:
What are its types?

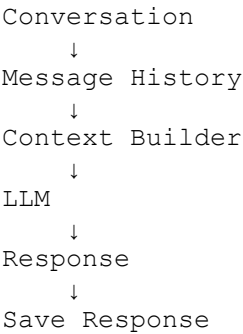
AI:
The main types are...

User:
Give me an example.

AI:
For example...

The previous messages are included as context when generating the next response.

Basic structure:



Step 4 — Add Conversation History

Create a sidebar:

Recent Chats

+ New Chat

Machine Learning
Python Interview
NLP Questions
Project Discussion
Research Paper

Each conversation should have:

Chat ID
Title
Messages
Created Time
Updated Time

The user should be able to:

- Create a new chat
- Open an old chat
- Delete a chat
- Rename a chat

Step 5 — Add Streaming Response

Instead of waiting for the complete answer:

Generating...

the application should display:

Artificial
Artificial Intelligence
Artificial Intelligence is
Artificial Intelligence is a field
Artificial Intelligence is a field of...

This gives the application a much more realistic ChatGPT-like experience.

Step 6 — Add Message Controls

For every AI response, add:

Copy
Regenerate

For user messages:

Edit

Example:

AI Response

Machine learning is a branch of AI...

[Copy] [Regenerate]

If the user selects **Regenerate**, generate another response using the same conversation context.

Step 7 — Add System Prompt

Add an optional settings panel:

System Instructions

You are a helpful AI assistant.
Explain concepts clearly with examples.

The system prompt controls the behavior of the assistant.

Example:

System:
You are an AI tutor.

User:
Explain Transformers.

AI:
A Transformer is...

Step 8 — Add Generation Controls

Project 1 already proposes controls such as Temperature, Top-K, Top-P and maximum new tokens.

Move these into Project 2's settings:

Temperature	[0.7]
Top-K	[50]
Top-P	[0.9]
Max Tokens	[512]

The user can experiment with how generation changes.

9. DOCUMENT CHAT — Major Upgrade

This should be one of the most important additions in Project 2.

Allow the user to upload:

PDF
TXT
DOCX

Example:

Upload:
research_paper.pdf

Then:

User:
What is the main objective of this paper?

The system searches the uploaded document and gives an answer based on the document.

10. Implement RAG

Use a simple Retrieval-Augmented Generation pipeline:

```
Document
↓
Extract Text
↓
Split into Chunks
↓
Create Embeddings
↓
```


Store in FAISS
↓
User Question
↓
Question Embedding
↓
Similarity Search
↓
Relevant Chunks
↓
LLM
↓
Answer

For example:

PDF:
100 pages

↓
Split into:
500 text chunks

↓
FAISS

↓
User:
"What methodology does the paper use?"

↓
Retrieve:
Top 5 relevant chunks

↓
LLM

↓
Answer

This is the key difference between a basic chatbot and a useful document-aware assistant.

11. Add Source References

When answering from a document, show where the answer came from.

Example:

Answer:

The proposed system uses a Transformer-based architecture...

Sources:

 research_paper.pdf
Page 4

 research_paper.pdf
Page 7

This makes the RAG system easier to verify.

12. Add Conversation Memory

Implement two levels of memory.

Short-term memory

Current conversation:

User → AI → User → AI

Long-term conversation history

Previously saved conversations:

Chat 1
Chat 2
Chat 3
Chat 4

The assistant should be able to reopen an old conversation.

13. Integrate LLM X-Ray

This is where Project 2 becomes an actual **upgrade** rather than simply "Project 1 + chatbot."

Add a button:

 Enable LLM X-Ray

Normal mode:

User
↓
AI Response

X-Ray mode:

User
↓
Tokens
↓
Token IDs
↓
Embeddings
↓
Transformer Layers
↓
Attention
↓
Hidden States
↓
Logits
↓
Probabilities
↓
Generated Tokens
↓
Response

These are exactly the internal stages already defined in Project 1.

14. X-Ray Panel

When the user clicks:

 X-Ray

show tabs:

Tokens | Embeddings | Layers | Attention | Hidden States | Logits

Tokens

What | is | artificial | intelligence | ?

Embeddings

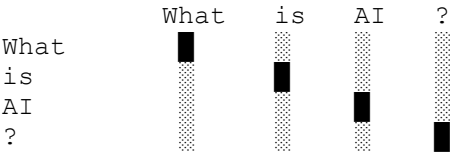
Show:

Token
Token ID
Embedding Dimension
Vector Statistics

Project 1 already specifies PCA visualization for embeddings.

Attention

Show:



Allow:

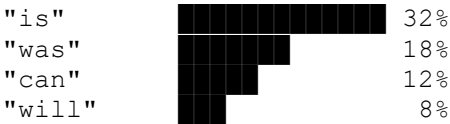
Layer: 12
Head: 4

Project 1 already defines layer/head selection for attention visualization.

15. Add Token Probability View

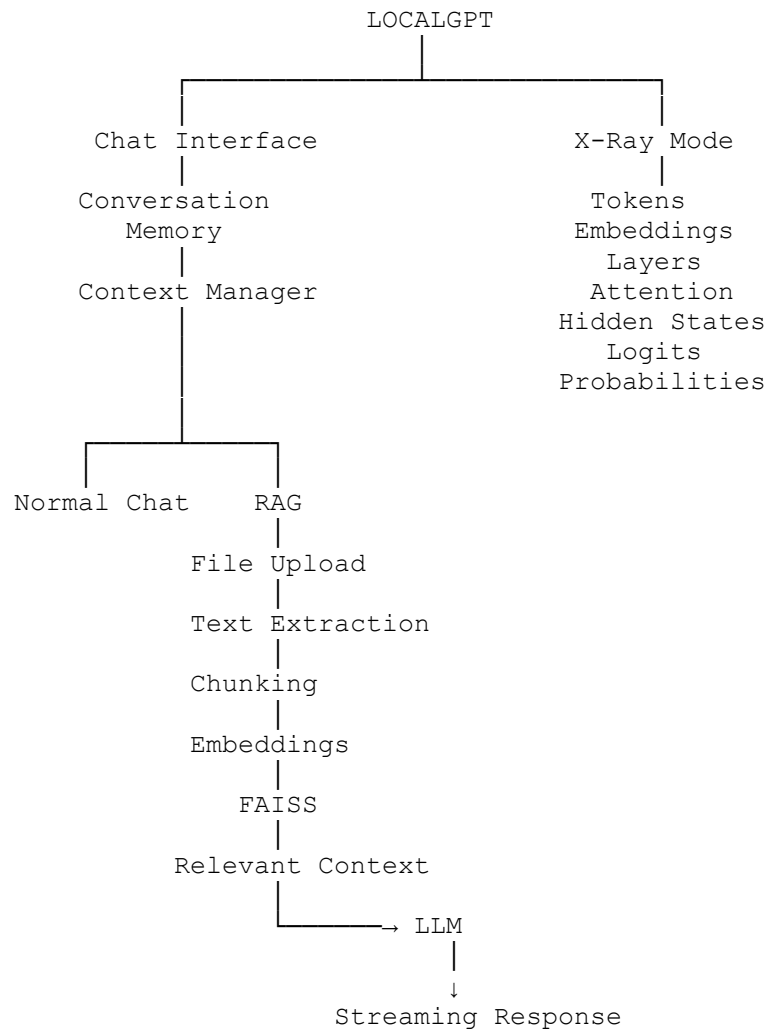
For the currently generated token:

Next Token Prediction



This extends the logits/probability visualization from Project 1.

16. Final Application Architecture



17. Final UI

The final application should look approximately like:

LOCALGPT		⚙ Settings
+ New Chat	User Explain Transformers	
Today	AI Transformers are neural network models...	
Transformers		
Python		
Research		
Documents	[Copy] [Regenerate] [📎 X-Ray]	
	User Explain with an example	
	AI Consider a translation system...	
📎 Upload	Ask anything...	▶

18. Expected Final Workflow

A user should be able to do this:

- 1. Open LocalGPT
- ↓
- 2. Start New Chat
- ↓
- 3. Ask a question
- ↓
- 4. Receive streaming response
- ↓
- 5. Ask follow-up question
- ↓
- 6. AI remembers conversation
- ↓
- 7. Upload PDF
- ↓
- 8. Ask questions about PDF
- ↓
- 9. Get answer + source
- ↓
- 10. Click X-Ray
- ↓
- 11. Inspect tokens
- ↓
- 12. Inspect embeddings
- ↓
- 13. Inspect attention
- ↓
- 14. Inspect logits
- ↓
- 15. Observe token generation

19. Project 1 → Project 2 Upgrade

Project 1 — LLM X-Ray	Project 2 — LocalGPT
Single prompt	Multi-turn conversation
Basic UI	ChatGPT-style UI
Generate response	Streaming response
Token visualization	Token visualization
Embedding visualization	Embedding visualization
Attention visualization	Attention visualization
Hidden states	Hidden states
Logits	Logits
Token generation	Token generation
Basic controls	Advanced settings
No document chat	PDF/DOCX/TXT chat
No retrieval	RAG
No history	Chat history
No memory	Conversation memory
X-Ray as main interface	X-Ray as advanced mode

20. Final Deliverable

The final deliverable is:

A locally running ChatGPT-style AI assistant that can:

Enter prompt
↓
Have a conversation
↓
Remember previous messages
↓
Generate streaming responses
↓
Upload documents
↓
Ask questions about documents
↓
Retrieve relevant information using RAG
↓
Show document sources
↓
Inspect LLM internals using X-Ray
↓
View tokens
↓
View embeddings
↓
View attention
↓
View logits/probabilities
↓
Observe token-by-token generation

So the progression is very clear:

Project 1:

"Understand what happens inside an LLM."

Project 2:

"Build a usable ChatGPT-like local assistant and allow the user to see what happens inside the LLM."

Please reach out if you have a doubt:

Mr. Yashwanth V S Devang

+91 9964778336



You are **Now**
someTHING..!
Different